

MODUL 1: TILE & DECK ENGINE (GO)

Mahjong Custom v1.0 — Panduan Arsitektur & Logika Koding untuk Pemula

Backend Game Logic • Go (Golang) • Core Engine

1. Gambaran Umum & Konsep Dasar

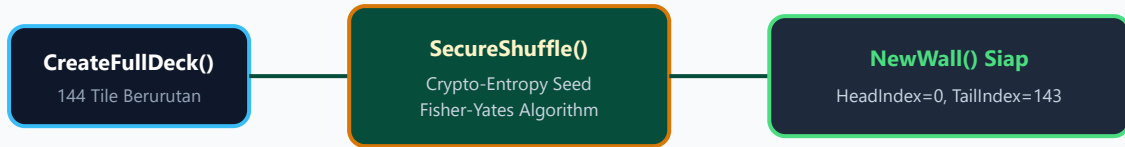
Pada Modul 1 ini, kita membangun pondasi paling mendasar dari game Mahjong, yaitu **representasi fisik 144 Tile**, **pengacakan aman (*secure shuffle*)**, serta **mekanisme pembagian kartu awal (*initial deal*)** dan **alokasi Dead Wall**.

Komposisi 144 Tile Fisik:

Kategori Suit	Simbol / Nilai	Jumlah Tiap Nilai	Total Tile
Bamboo (Bambu)	Nomor 1 s.d. 9	4 keping	36 Tile
Dot (Lingkaran)	Nomor 1 s.d. 9	4 keping	36 Tile
Character (Karakter)	Nomor 1 s.d. 9	4 keping	36 Tile
Wind (Angin)	East, South, West, North	4 keping	16 Tile
Dragon (Naga)	Red, Green, White	4 keping	12 Tile
Flower (Bunga)	Plum, Orchid, Chrysanthemum, Bamboo	1 keping	4 Tile
Season (Musim)	Spring, Summer, Autumn, Winter	1 keping	4 Tile
TOTAL KESELURUHAN DECK			144 Tile

2. Diagram Flowchart Logika Engine

Flowchart 1: Alur Inisialisasi Wall & Pengacakan (Shuffle)



Flowchart 2: Alur Pembagian Kartu Awal (Initial Deal) & Alokasi Wall



3. Penjelasan Koding Baris per Baris

A. Struktur Data Tile (server/pkg/engine/tile.go)

```
type Tile struct {
    ID      int   `json:"id"`           // ID unik 0-143 untuk setiap keping fisik
    Suit    Suit  `json:"suit"`         // Bamboo, Dot, Character, Wind, Dragon, Flower, Season
    Value   int   `json:"value"`        // Angka 1-9 atau ID simbol angin/naga
    IsBonus bool  `json:"is_bonus"`     // True jika Flower atau Season
}
```

Mengapa Perlu Field ID Unik?

Meskipun dalam permainan Mahjong ada 4 keping "5-Bamboo" yang tampak identik, di server setiap keping memiliki ID unik (misal ID 16, 17, 18, 19). Hal ini penting untuk melacak riwayat pergerakan fisik setiap batu dari wall ke tangan, discard pond, atau meld terbuka tanpa resiko bug duplikasi.

B. Pengacakan Aman Fisher-Yates (server/pkg/engine/deck.go)

```
// Mengambil 8 byte acak dari sistem operasi sebagai seed kriptografi
var seedBytes [8]byte
rand.Read(seedBytes[:])
seed := int64(binary.LittleEndian.Uint64(seedBytes[:]))

// Mengacak urutan array dengan Fisher-Yates
for i := len(shuffled) - 1; i > 0; i-- {
    j := rng.Intn(i + 1)
```

```
    shuffled[i], shuffled[j] = shuffled[j], shuffled[i]  
}
```

Kenapa Tidak Menggunakan Random Biasa?

Fungsi `math/rand` biasa menggunakan seed waktu (timestamp) yang bisa diprediksi oleh hacker. Dengan mengambil sumber entropi dari `crypto/rand`, urutan kartu menjadi 100% acak, adil, dan mustahil dimanipulasi dari sisi client/pemain.

C. Pointer Pengambilan Tile (Head vs Tail)

```
// Draw normal mengambil dari depan (Head)
func (w *Wall) DrawTile() (Tile, error) {
    tile := w.Tiles[w.HeadIndex]
    w.HeadIndex++ // Geser penunjuk ke kanan
    return tile, nil
}

// Replacement Draw (Flower / Kong) mengambil dari belakang (Tail)
func (w *Wall) DrawReplacementTile() (Tile, error) {
    tile := w.Tiles[w.TailIndex]
    w.TailIndex-- // Geser penunjuk ke kiri
    return tile, nil
}
```

💡 Insight Teknis untuk Pemula:

Teknik menggunakan **HeadIndex** dan **TailIndex** ini disebut *Pointer Array*. Keunggulannya adalah sangat hemat memori (RAM) dan tidak memerlukan penghapusan elemen array (*slice reallocation*), sehingga eksekusi draw berlangsung secepat kilat (0 nanodetik).

4. Hasil Pengujian Otomatis (Unit Test)

Unit test di file `server/pkg/engine/tile_test.go` memvalidasi:

- ✅ **144 Tile Terbentuk Lengkap** tanpa ada yang terlewat (36 Bamboo, 36 Dot, 36 Character, 16 Wind, 12 Dragon, 4 Flower, 4 Season).
- ✅ **Tidak Ada Duplikasi ID** (setiap tile memiliki nomor unik 0 s.d. 143).
- ✅ **Pembagian Dealer Tepat 14 Tile** dan 3 pemain lainnya masing-masing tepat 13 tile.
- ✅ **Sisa Tile Live Wall Tepat 77 Tile** setelah disisihkan 14 Dead Wall.